

PRIMO TENTATIVO DI FORMALIZZAZIONE DEL PROBLEMA

Versione: aprile 2026

Problema: Dato un deployment con il suo score di sicurezza e un budget B, trovare la combinazione di azioni di miglioramento che massimizza l'aumento del punteggio di sicurezza rispettando il budget.

ALGORITMO GREEDY

INPUT:

D: deployment corrente

A: catalogo azioni $\{(a, \text{cost}(a), \text{effect}(a))\}$

B: budget disponibile

OUTPUT:

S: insieme di azioni selezionate

D': deployment migliorato

Δ score: miglioramento ottenuto

Pseudocodice

BEGIN:

1. $S \leftarrow \emptyset$

2. $\text{budget_rimanente} \leftarrow B$

3. $D_corrente \leftarrow D$

4. $\text{score_corrente} \leftarrow \text{SecFog}(D_corrente)$

5. $A_disponibili \leftarrow A$

6. RIPETI

a. $\text{best_action} \leftarrow \text{null}$

$\text{best_ratio} \leftarrow 0$

b. FOR ogni azione $a \in A_disponibili$:

IF $\text{cost}(a) \leq \text{budget_rimanente}$ THEN

$D_temp \leftarrow \text{applica}(D_corrente, \text{effect}(a))$

$\text{score_temp} \leftarrow \text{SecFog}(D_temp)$

$\Delta t \leftarrow \text{score_temp.t} - \text{score_corrente.t}$

$\text{ratio} \leftarrow \Delta t / \text{cost}(a)$

IF $\text{ratio} > \text{best_ratio}$ THEN

$\text{best_action} \leftarrow a$

```

    best_ratio ← ratio
  END IF

END IF

END FOR

c. IF best_action = null THEN
  BREAK // nessuna azione migliorativa nel budget
END IF

d. S ← S ∪ {best_action}
e. budget_rimanente ← budget_rimanente - cost(best_action)
f. D_corrente ← applica(D_corrente, effect(best_action))
g. score_corrente ← SecFog(D_corrente)
h. A_disponibili ← A_disponibili \ {best_action}

7. FINCHE' budget_rimanente = 0 OR A_disponibili = ∅

8. RETURN S, D_corrente, score_corrente.t - SecFog(D).t

```

Complessità totale: $O(n^2)$ chiamate a SecFog (collo di bottiglia)

Note dell'incontro del 16/04/2026:

Per il prossimo incontro:

- implementare buy e improve
- decisione su due livelli:
 - sul deployment
 - su singolo nodo
- implementare la matrice dei costi (fisso) e del gain (calcolato con SecFog)
- provare a implementare tre algoritmi:
 - greedy
 - forza bruta
 - MILP
 e confrontare i risultati
- decidere il linguaggio/framework per implementare l'algoritmo (C, Python, Java) x (GUROBI o CPLEX o OR-TOOLS),
- usare SecFog in combinazione col mio algoritmo per valutare il dispiegamento baseline contro quello "ottimizzato" dell'algoritmo.